Bloco · Tipos de teste · padrão FGV (15 questões)

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais.
>
> Assunto único: teste de software (fila nº 5). Níveis, alfa × beta, caixa-preta × caixa-branca, valor-limite, partição de equivalência, cobertura, complexidade ciclomática, carga × estresse, regressão, erro/defeito/falha, stub × driver, princípios, estático × dinâmico, pirâmide e TDD. Marque "?" no que não souber e nunca deixe em branco.

Questão 1

A equipe de TI de um Tribunal de Justiça está concluindo um novo módulo de distribuição de processos. Durante o projeto, foram realizadas as seguintes atividades de teste:

I. o desenvolvedor verificou, de forma isolada, o método que calcula o prazo de redistribuição, substituindo o acesso ao banco de dados por um objeto simulado;

II. a equipe verificou se o módulo de distribuição envia corretamente os dados do processo ao módulo de intimações, por meio da API interna;

III. a equipe de qualidade executou, em ambiente de homologação semelhante ao de produção, testes do sistema completo, incluindo os requisitos de tempo de resposta;

IV. servidores das varas, usuários finais do sistema, verificaram se ele atende às necessidades do trabalho cartorário antes da implantação.

As atividades I, II, III e IV correspondem, respectivamente, aos níveis de teste:

(A) integração, unitário, aceitação e sistema.

(B) unitário, sistema, integração e aceitação.

(C) unitário, integração, sistema e aceitação.

(D) unitário, integração, aceitação e sistema.

(E) integração, integração, sistema e regressão.

Questão 2

Um Tribunal de Justiça desenvolveu um aplicativo para que advogados consultem a pauta de audiências. Antes do lançamento, o aplicativo passou por duas etapas de teste com usuários:

- na primeira, servidores convidados foram à sede da Secretaria de TI e usaram o aplicativo sob a observação dos desenvolvedores, que anotavam as dificuldades encontradas;
- na segunda, uma versão de pré-lançamento foi disponibilizada a um grupo de advogados, que a utilizaram em seus próprios escritórios, sem acompanhamento da equipe, e reportaram os problemas por um formulário.

As duas etapas descritas correspondem, respectivamente, a:

(A) teste alfa e teste beta.

(B) teste beta e teste alfa.

(C) teste de sistema e teste de aceitação contratual.

(D) teste alfa e teste de regressão.

(E) teste de usabilidade e teste de integração.

Questão 3

O sistema de peticionamento eletrônico de um Tribunal de Justiça aceita a juntada de documentos com quantidade de páginas entre 1 e 300, inclusive. A quantidade de páginas é sempre um número inteiro.

Aplicando-se a técnica de análise de valor-limite, com dois valores por fronteira, o conjunto de entradas a ser testado é:

(A) 1, 150 e 300.

(B) 0, 150 e 301.

(C) 1 e 300.

(D) 0, 1, 300 e 301.

(E) 0, 1, 2, 299, 300 e 301.

Questão 4

O sistema de cadastro de conciliadores de um Tribunal de Justiça aceita candidatos com idade entre 21 e 65 anos, inclusive. A idade é informada sempre como número inteiro.

Utilizando a técnica de particionamento em classes de equivalência, considerando classes válidas e inválidas e um caso de teste para cada classe, a quantidade mínima de casos de teste para o campo idade é:

(A) 2.

(B) 3.

(C) 4.

(D) 6.

(E) 45.

Questão 5

Sobre as técnicas de teste de software, analise as afirmativas a seguir.

I. Na técnica de caixa-branca, os casos de teste são derivados da estrutura interna do código, como na busca por cobertura de comandos e de decisões.

II. A análise de valor-limite e a tabela de decisão são técnicas de caixa-branca, pois exigem que o testador conheça as condições escritas no código-fonte.

III. Na técnica de caixa-preta, o testador pode projetar os casos de teste a partir da especificação, sem acesso ao código-fonte.

Está correto o que se afirma em

(A) I, apenas.

(B) II, apenas.

(C) I e II, apenas.

(D) II e III, apenas.

(E) I e III, apenas.

Questão 6

Considere a função a seguir, escrita em Python, que calcula a taxa de expedição de uma certidão:

```
def taxa_certidao(valor, gratuidade):
    taxa = valor * 0.01
    if gratuidade:
        taxa = 0
    return taxa
```

A equipe de testes executou um único caso de teste: `taxa_certidao(1000, True)`.

Com esse único caso, é correto afirmar que foi obtida

(A) 100% de cobertura de comandos e 100% de cobertura de decisões.

(B) 100% de cobertura de comandos e 50% de cobertura de decisões.

(C) 75% de cobertura de comandos e 50% de cobertura de decisões.

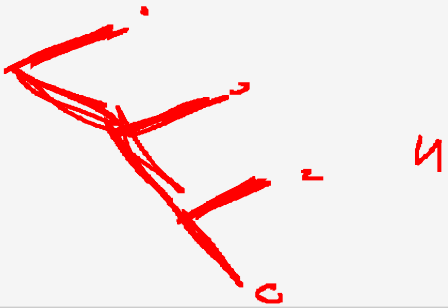
(D) 50% de cobertura de comandos e 100% de cobertura de decisões.

(E) 100% de cobertura de decisões, o que garante 100% de cobertura de caminhos.

Questão 7

Considere a função a seguir, escrita em Python, que classifica a situação de um processo:

```
def classificar(dias, urgente):
    if urgente:
        return "PRIORITARIO"
    if dias > 100:
        return "ATRASADO"
    elif dias > 30:
        return "ATENCAO"
    return "NO PRAZO"
```



A complexidade ciclomática dessa função, que indica o número mínimo de caminhos independentes a serem testados, é igual a:

(A) 2.

(B) 3.

(C) 4.

(D) 5.

(E) 6.

Questão 8

No último dia de um prazo processual de grande repercussão, um Tribunal de Justiça estima que até 8.000 advogados acessem simultaneamente o sistema de peticionamento. Antes dessa data, a equipe de TI realizou dois testes:

- no primeiro, simulou 8.000 usuários simultâneos, para verificar se o tempo de resposta se mantinha dentro do limite definido nos requisitos;
- no segundo, aumentou progressivamente a quantidade de usuários simulados até 30.000, bem acima da capacidade prevista, para observar como o sistema falha e se ele se recupera depois.

Os dois testes descritos são, respectivamente, de:

(A) carga e estresse.

(B) estresse e carga.

(C) regressão e volume.

(D) aceitação e pico.

(E) usabilidade e resistência.

Questão 9

Um defeito no cálculo de prazos em dobro para a Defensoria Pública foi corrigido no sistema processual de um Tribunal de Justiça. Em seguida, a equipe de testes:

- executou novamente o caso de teste que havia revelado o defeito, para confirmar que ele foi de fato corrigido;
- reexecutou a suíte automatizada das demais funcionalidades (intimações, distribuição e certidões), para verificar se a alteração não introduziu defeitos no que já funcionava.

As duas atividades descritas são, respectivamente, teste de:

(A) regressão e teste de confirmação.

(B) unidade e teste de integração.

(C) fumaça (smoke) e teste de sanidade.

(D) aceitação e teste de regressão.

(E) confirmação (reteste) e teste de regressão.

Questão 10

Ao especificar o cálculo de prazos de um sistema processual, um analista entendeu equivocadamente que o prazo deveria ser contado em dias corridos, e não em dias úteis. Com base nesse entendimento, o desenvolvedor implementou a contagem em dias corridos no código-fonte. Após a implantação, um advogado consultou o sistema e viu, na tela, uma data de vencimento anterior à correta.

Segundo a terminologia de teste de software, o entendimento equivocado do analista, a contagem incorreta no código-fonte e a data errada exibida ao advogado são, respectivamente:

(A) falha, defeito e erro.

(B) defeito, erro e falha.

(C) erro, falha e defeito.

(D) erro, defeito e falha.

(E) defeito, falha e erro.

Questão 11

Uma equipe adota a integração incremental descendente (top-down) no sistema de custas de um Tribunal de Justiça. O módulo de distribuição, no topo da hierarquia, já está pronto, mas o módulo de cálculo de custas, que é chamado por ele, ainda não foi implementado.

Para testar o módulo de distribuição, a equipe deve criar um

(A) driver, que simula o módulo de cálculo de custas e devolve respostas predefinidas ao módulo de distribuição.

(B) stub, que simula o módulo de cálculo de custas, devolvendo respostas predefinidas quando chamado pelo módulo de distribuição.

(C) driver, que simula um módulo superior responsável por chamar o módulo de distribuição.

(D) stub, que chama o módulo de distribuição e verifica as respostas por ele produzidas.

(E) oráculo de teste, que substitui definitivamente o módulo de cálculo de custas em produção.

Questão 12

Sobre os princípios do teste de software, assinale a afirmativa **incorreta**.

(A) O teste pode demonstrar a presença de defeitos, mas não pode provar a sua ausência.

(B) O teste exaustivo, com todas as combinações de entradas e pré-condições, é inviável, exceto em casos triviais; por isso, usam-se análise de risco e técnicas de teste para priorizar.

(C) Repetir sempre os mesmos testes tende a fazer com que eles deixem de encontrar defeitos novos, sendo necessário revisá-los periodicamente.

(D) Um sistema no qual não foram encontrados defeitos após testes extensivos certamente atende às necessidades e expectativas dos usuários.

(E) Os defeitos tendem a se concentrar em um pequeno número de módulos do sistema.

Questão 13

A Secretaria de TI de um Tribunal de Justiça realizou as seguintes atividades de garantia da qualidade em um novo sistema:

I. inspeção do documento de requisitos por analistas e usuários-chave;

II. análise do código-fonte por uma ferramenta que aponta variáveis não utilizadas e trechos duplicados;

III. execução da suíte de testes unitários, com medição de cobertura;

IV. revisão do código pelos pares antes da aprovação de um pull request.

São exemplos de teste estático as atividades

(A) I e III, apenas.

(B) II e III, apenas.

(C) III e IV, apenas.

(D) I, II, III e IV.

(E) I, II e IV, apenas.

Questão 14

Um Tribunal de Justiça está adotando integração contínua e automação de testes no desenvolvimento do seu sistema processual.

Sobre a automação de testes e a pirâmide de testes, assinale a afirmativa correta.

(A) A base da pirâmide deve concentrar a maior quantidade de testes, os unitários, por serem rápidos e baratos; os testes de interface ponta a ponta ficam em menor número no topo, por serem lentos e frágeis.

(B) A maior parte da suíte deve ser composta por testes de interface ponta a ponta, por reproduzirem fielmente o uso real e serem mais estáveis que os testes unitários.

(C) Os testes de regressão são maus candidatos à automação, pois precisam ser reescritos a cada nova versão do sistema.

(D) Com a automação, os testes manuais e exploratórios tornam-se dispensáveis, pois a suíte automatizada cobre todos os cenários possíveis.

(E) Na integração contínua, os testes automatizados devem ser executados apenas antes da implantação em produção, para não atrasar os commits da equipe.

Questão 15

Uma equipe que pratica TDD está desenvolvendo a classe `CalculadoraPrazo`, que soma dias úteis a uma data, ignorando sábados, domingos e feriados. A consulta de feriados é feita por um serviço externo do Tribunal. Antes de implementar a classe, a equipe escreveu o teste a seguir, em Python:

```
class FeriadosStub:
    def eh_feriado(self, data):
        return data in {"2026-11-20"}

def test_prazo_pula_feriado():
    calc = CalculadoraPrazo(FeriadosStub())
    # 19/11/2026 é quinta-feira; 20/11 é feriado
    assert calc.vencimento("2026-11-19", 1) == "2026-11-23"
```

Sobre esse teste, é correto afirmar que

(A) se trata de um teste de integração, pois a `CalculadoraPrazo` interage com outra classe durante a execução.

(B) `FeriadosStub` é um mock, pois verifica quantas vezes o método `eh_feriado` foi chamado e com quais parâmetros.

(C) `FeriadosStub` isola a unidade do serviço real de feriados, fornecendo respostas predefinidas, o que torna o teste unitário rápido e determinístico.

(D) pelo TDD, o teste só poderia ter sido escrito depois de implementada a `CalculadoraPrazo`, para que o primeiro resultado da execução fosse verde.

(E) como o `assert` apenas compara datas, trata-se de um teste estático, que não exige a execução do código.

Minhas respostas

Questão	Resposta
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	